DATE:__/__/____ DAY:__/__/____

M. Hammad-ud-din
SAP # 65227

Analysis of Algorithm

Assignment # 01

Q1: Find the largest and smallest Elements?

1):- **Algorithmic Strategy:**

We use a linear scanning approach. Initially, we assume that the first element of the array is both the smallest and largest element. Then we compare each remaining element with the current smallest and largest values. If the current element is smaller than smallest, we update smallest. If it is larger than largest, we update largest. The algorithm scans the array only once, so its running time grows linearly with the input size.

2):- **Pseudocode:**

Input: Array A of n integers
output: smallest and largest values or elements

```
smallest = 1-A[0]
Largest = A[0]
For i = 1 to N-1
```

DATE:__/__/____ DAY:__/__/____

```
        if A[i] < smallest then
            smallest = A[i]
        end if
        if A[i] > Largest then
            Largest = A[i]
        end if
    end for
    return smallest, Largest
End Algorithm-
```

3)- Major Primitive Operations

The main primitive operations are:

1)- Accessing an array element A[i]-
2)- Comparing A[i] with smallest-
3)- Comparing A[i] with largest-
4)- updating smallest-
5)- updating largest-
6)- Incrementing the loop variable i-

The numbers of comparisons determine the running time for each element (except the first): at most two comparisons are performed so, total comparisons -

$$\approx 2(n-1)$$

DATE:__/__/____ DAY:__/__/____

4)- Running Time:

Best case →

The algorithm must check all element even in the best case, so the running time is $\Theta(n)$.

Worst case →

The algorithm also checks all elements, So the running time is $\Theta(n)$.

5)- Asymptotic Notation →

The running time is Linear in the size of the input -

$O(n)$, $\Omega(n)$, $\Theta(n)$

The tight bound is : $\Theta(n)$

6)- Space Complexity:

The algorithm uses only a few extra variables (smallest, largest, i) No additional array or data structure is required.

Space complexity = $O(1)$.

Example :- A := [12, 5, 18, 3, 9] min = 12, max = 12

i	A[i]	A[i] < min ?	A[i] > max ?	min	max
1	12	NO	NO	12	12
2	5	Yes	NO	5	12
3	18	NO	Yes	5	18
4	3	Yes	NO	3	18
5	9	NO	NO	3	18

Scanned with CamScanner

DATE:__/__/____ DAY:__/__/____

Final Answer:

Min = 3

Max = 18

Q2: Search for a Given value x

Given n integers, design an algorithm to determine whether a particular value x exists in the array we will use two different algorithm:

1)- Linear Search

2)- Binary Search.

7)- Linear Search:

1)- checks elements one by one from the beggining.

2)- If an element equal x, return True.

3)- If all elements are checked and x is not found, return false.

4)- Works on both sorted and unsorted arrays.

b)- Binary search:

1)- Compares x with the middle element.

2)- If equal, Return True.

3)- If x is smaller, Search in left half.

Scanned with CamScanner

DATE:__/__/____ DAY:__/__/____

4) If x is larger, search in right half-
5) works only on sorted arrays-

6) Pseudocode for both Algorithms

Linear:

```
Algorithm Linear search (A, n, x)
    Input : Array A of n integers and value x
    output : TRUE if x is found, otherwise
    False -

for i = 0 to n-1 do

    if A[i] = x then
        return True
    end if
end for
    Return FALSE

End Algorithm -
```

Binary:

```
Algorithm Binary search (A, n, x)
    Input : sorted array if A of n integers
                and value x
output: TRUE if x is found, otherwise FALSE
    Low = 0
    high = n-1
while Low ≤ high do
```

DATE:__/__/____ DAY:__/__/____

: Mid = (Low + high) / 2

if A [mid] = x then
Return TRUE
else if x < A [Mid] then
high = Mid - 1
else
low = Mid + 1
end if
end while
Return FALSE

End Algorithm -

9) Best case & worst case Time complexity :

a) Linear :

: x is at the first position
$T_{best}(n) = O(1)$ (one comparison)

: x is at the last position or not present -

$T_{worst}(n) = O(n)$ (Up to n comparison)

DATE:__/__/____ DAY:__/__/____

(b). Binary:

: x is at the middle element

$T_{best}(n) = \Omega(1)$

Search area is divided in half repeatedly -

$T_{worst}(n) = O(\log n)$ -

10). Big-O, Big-Ω and Big-Θ

Algorithm	Best case	worst case	Big-O	Big-Ω	Big-Θ
Linear search	$O(1)$	$O(n)$	$O(n)$	$\Omega(1)$	$\Theta(n)$
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$	$\Omega(1)$	$\Theta(\log n)$

11). Comparison of the two Algorithms:

Linear Search	Binary Search
Simple and easy to implement	Faster for Large and sorted arrays.
works on sorted or unsorted arrays	Requires sorted arrays
Better for small arrays	Divides search space into two halves.
worst case time : $O(n)$	worst case time : $O(\log n)$

Scanned with CamScanner

DATE:__/__/____ **Example:** DAY:__/__/____

Example: Linear search:

Step	A[i]	Comparison	Result
1	4	4=23?	No
2	8	8=23?	No
3	15	15=23?	No
4	19	19=23?	No
5	23	23=23?	Yes - found

Binary search:

Step	Low	high	Mid	A[mid]	comparison	Action
1	0	8	3	19	23>19	Search Right
2	4	16	5	27	23<27	search left
3	4	4	4	23	23=23	Found -

Which Algorithm is more Efficient

For small or unsorted array, Linear search is a good choice because it is simple and does not require Sorting - For a large sorted array, Binary search is more efficient because its running time grows Logarithmically instead of Linearly -

So Binary search is more efficient for large and sorted arrays -

Scanned with CamScanner

DATE:__/__/____ DAY:__/__/____

(2). Effect of Input size & Arrangement:
The choice of algorithm depends on the size and arrangments of input data.-

(a):- Small input:
Linear search can be used because only a few elements needs to be checked.-

b):- Large input:
Binary search is better for large sorted arrays- Because its reduce the search area by half at each step-

(c):- Unsorted input:
Linear search can be used directly-
Binary search cannot be used unless the array is sorted-

(d):- Position of x:
In Linear search: beggining = fast, end = many checks, not present and all elements checked-

In binary search: Position of x is less important because the Search space is divided at each step-

DATE:__/__/____ DAY:__/__/____

Final Comparison

Feature	Linear search	Binary search
Array Requirments	Sorted or unsorted	Sorted
Best case	O(1)	O(1)
Worst case	O(n)	O(Log n)
simple to implement	Yes	Yes
small/unsorted Data	Suitable	Not directly suitable
Large Sorted Data	Less efficient	More efficient.

Conclusion :

Both algorithms can be used to determined wheather x exists in an array. The choice depends on the size and arrangments of the input data.